Aula 09 — Automação de Workflows com n8n

Etapa 2 — Aplicação, publicação e automações
Carga horária: 3 horas (teoria e prática)
Autores: Benevid Felix da Silva

Objetivos

Ao final desta aula, você será capaz de:

- Situar a automação de fluxos de trabalho na literatura de gestão de processos, distinguindo workflow de agente e integração por API de automação robótica de processos
- Instalar o n8n em contêiner e identificar as áreas da interface e do editor de workflows
- Construir um workflow que consulta uma API pública, decide por uma condição e registra o resultado em uma Data table, usando expressões para referenciar dados de outros nós
- Agendar um workflow com fuso horário definido, tratar falhas de consulta por uma saída de erro e acompanhar as execuções de produção
- Exportar workflows como arquivos JSON e operá-los pela linha de comando e pela API pública
- Registrar onde o workflow guarda dados pessoais e por quanto tempo

---

1. Antes da aula

Esta seção é feita em casa e leva cerca de 20 minutos, a maior parte de download: a imagem do n8n ocupa cerca de 1 GB. Ela usa o Docker instalado na Aula 02 e o Compose já usado na Aula 07, seção 4.3. Quem chegar com o n8n no ar e a conta criada começa a aula direto pela seção 2.

1.1 A pasta do projeto

Todo o trabalho da aula fica numa pasta `aula09`, versionada com git. Crie a pasta e copie para ela o conteúdo de `arquivos/aula09-n8n/` do material da disciplina, **incluindo os arquivos ocultos** `.env.exemplo` e `.gitignore`:

```
mkdir aula09 && cd aula09
cp -R <material>/arquivos/aula09-n8n/. .
```

```
git init
ls -A
```

A listagem deve mostrar:

```
.env.exemplo  .git  .gitignore  LEIA-ME.md  cliente_n8n.py  docker-
compose.yml  pedidos-fornecedores.json  workflows
```

e `ls workflows` deve listar os três arquivos `workflow01-consulta-cep.json`, `workflow02-rotina-diaria.json` e `workflow03-formulario-fornecedor.json`.

1.2 Subir o n8n

O `docker-compose.yml` da pasta descreve um único serviço:

```
services:
  n8n:
    image: n8nio/n8n:2.42.5          # versão exata: a interface muda
entre versões
    container_name: n8n
    restart: unless-stopped
    ports:
      - "127.0.0.1:5678:5678"        # só esta máquina acessa o editor
    environment:
      - GENERIC_TIMEZONE=America/Cuiaba   # fuso dos agendamentos
      - TZ=America/Cuiaba
    volumes:
      - n8n_data:/home/node/.n8n     # banco, chave de cifra e
execuções

volumes:
  n8n_data:
```

Três escolhas do arquivo evitam problemas adiante. A primeira é a **versão exata da imagem**: a documentação do n8n desaconselha as etiquetas móveis `latest` e `stable`, porque uma atualização muda a interface sem aviso, e todas as telas desta apostila são da versão 2.42.5. A segunda é a variável `GENERIC_TIMEZONE`, que define o fuso dos agendamentos; sem ela, o n8n adota o fuso de Nova York, e uma rotina marcada para as 8h roda às 8h ou às 9h de Cuiabá, conforme a época do ano, porque Nova York adota horário de verão e Cuiabá não. A terceira é o mapeamento `127.0.0.1:5678:5678`, que, como na Aula 07, seção 3.1, publica a porta só para a própria máquina. Os dados ficam no volume `n8n_data`, separado do contêiner, e sobrevivem a um `docker compose down`.

Suba o serviço e verifique:

```
docker compose up -d
curl -s http://127.0.0.1:5678/healthz
docker compose logs n8n | grep "Version:"
```

A resposta do `curl` deve ser `{"status":"ok"}` e o log deve conter `Version: 2.42.5`. Nos primeiros segundos depois do `up`, o `curl` pode não responder; espere alguns segundos e repita.

> **Compose no macOS com Homebrew.** O Homebrew instala o Compose, mas não o registra no Docker, e o `docker compose` responde `unknown command`. O registro é um atalho na pasta de plugins:
>
> ```
> brew install docker-compose
> mkdir -p ~/.docker/cli-plugins
> ln -sfn /opt/homebrew/lib/docker/cli-plugins/docker-compose
> ~/.docker/cli-plugins/docker-compose
> docker compose version
> ```
>
> A última linha deve responder `Docker Compose version` seguido do número da versão.

> **Sem Docker.** Quem ainda não tem o Docker funcionando pode instalar o n8n pelo npm, com o Node.js 24 ou superior: `npm install n8n -g` e `n8n start`, com `export GENERIC_TIMEZONE=America/Cuiaba` antes. A documentação avisa que esse caminho é destinado a desenvolvimento e será descontinuado no n8n 3.0. Quem seguir por ele pula esta seção e, na seção 8.2, digita os comandos sem o prefixo `docker compose exec n8n`.

1.3 A conta do proprietário

Abra `http://localhost:5678` no navegador. Na primeira abertura, o n8n pede a criação da conta do **proprietário** (*owner*), o administrador da instalação: a tela **Set up owner account** solicita e-mail, nome, sobrenome e uma senha de no mínimo oito caracteres, com ao menos um número e uma letra maiúscula. Em seguida aparecem um questionário de personalização e a oferta de uma chave de licença gratuita, mediante cadastro de e-mail; nenhum dos dois é necessário nesta aula, e ambos podem ser dispensados.

A conta fica só na sua instalação, e o e-mail não precisa ser verificado. A verificação desta etapa é chegar à tela **Overview**, descrita na seção 4.1, com a lista de workflows vazia.

1.4 O ambiente Python do cliente da API

A seção 8.3 usa o `cliente_n8n.py`, que depende de duas bibliotecas. Crie o ambiente virtual dentro de `aula09`, como na Aula 04, seção 3.2:

```
python3 -m venv .venv
source .venv/bin/activate
pip install httpx python-dotenv
python -c "import httpx, dotenv; print('ok')"
```

A última linha deve imprimir `ok`. No Caminho Wd, use `python -m venv .venv` e `.venv\Scripts\Activate.ps1`.

Feche a etapa com o primeiro commit; o `.gitignore` copiado já exclui `.env` e `.venv/`:

```
git add . && git commit -m "Projeto da Aula 09: n8n em contêiner"
git status --short
```

O `git status --short` não deve listar nada.

---

2. Automação de processos e fluxos de trabalho

O exemplo que atravessa a aula é o de um setor hipotético que recebe pedidos de cadastro de fornecedores, cada um com razão social, CNPJ, representante, e-mail e o CEP da sede. Um servidor repete à mão o mesmo roteiro para cada pedido: consulta o CEP para confirmar o município e o estado, copia os dados para uma planilha e redige ao representante um aviso com o resultado. Os passos não exigem julgamento, e sim consulta e transcrição, e é essa regularidade que torna o roteiro um bom candidato à automação.

2.1 Workflow e o modelo de referência

Um **processo de negócio** é o encadeamento de atividades que uma organização executa para produzir um resultado. A Workflow Management Coalition (WfMC), entidade que padronizou a área nos anos 1990, define **workflow** como a automação de um processo de negócio, no todo ou em parte, durante a qual documentos, informações ou tarefas passam de um participante a outro para que atuem sobre eles, segundo um conjunto de regras de procedimento. O participante pode ser uma pessoa ou um sistema, e é essa equivalência que permite automatizar um passo sem redesenhar o processo inteiro.

O modelo de referência da WfMC, descrito por Hollingsworth (1995), separa componentes que costumam vir misturados na prática. A tabela a seguir os relaciona com o que cada um é no n8n, apresentado na seção 3:

Componente	O que é	No n8n
Definição do processo	A descrição dos passos, das condições e dos dados que circulam	O workflow desenhado no editor
Serviço de execução (*workflow enactment service*)	O programa que interpreta a definição e a executa caso a caso	O motor de execução
Aplicações invocadas	Os sistemas externos que cada passo aciona	Os nós de integração, como o HTTP Request
Administração e monitoramento	O registro do que cada execução fez	A área **Executions**

Essa separação explica por que toda ferramenta de automação tem um editor, um motor e um histórico de execuções.

2.2 Padrões de workflow

A forma de descrever os processos foi sistematizada por van der Aalst et al. (2003), que catalogaram os **padrões de workflow**, construções recorrentes de controle de fluxo encontradas em qualquer linguagem de processos. Os cinco padrões básicos bastam para o cadastro de fornecedores:

Padrão	Comportamento	Onde aparece nesta aula
Sequência	Uma atividade só começa quando a anterior termina	Do gatilho até a decisão, seção 6.2
Escolha exclusiva	Uma condição envia o caso a um entre vários caminhos	O nó If, seção 6.4
Junção simples	Caminhos alternativos voltam a um ponto comum sem esperar uns pelos outros	O nó de registro, seções 6.2 e 7.4
Divisão paralela	Um caso segue por vários caminhos ao mesmo tempo	Não usado nesta aula
Sincronização	Caminhos paralelos esperam uns pelos outros antes de prosseguir	Não usado nesta aula

O catálogo serve como vocabulário para avaliar uma ferramenta, porque permite perguntar, para cada padrão, como ela o expressa e como o executa.

2.3 Integração por API e automação robótica

Um passo automatizado alcança os sistemas que o processo usa de duas maneiras. A **integração por API** chama diretamente o serviço que guarda o dado, por uma

requisição que devolve o resultado em formato estruturado; Hohpe e Woolf (2003) descrevem essa ligação entre sistemas como integração de aplicações, com padrões próprios de mensagem, transformação e roteamento. A **automação robótica de processos** (RPA, *Robotic Process Automation*), segundo van der Aalst, Bichler e Heinzl (2018), age de fora para dentro, imitando o que uma pessoa faz na tela, e por isso é a saída quando o sistema não oferece API.

	Integração por API	**RPA**
Como alcança o sistema	Requisição ao serviço, resposta estruturada	Cliques e digitação na interface gráfica
Ponto forte	Estabilidade: não quebra quando a tela muda	Alcance: funciona com qualquer sistema que tenha tela
Ponto fraco	Exige que o sistema ofereça API	Quebra a cada mudança de layout

O n8n trabalha pela integração por API, e todo o workflow desta aula usa esse caminho.

2.4 Workflow e agente

Um agente decide a cada passo o que fazer: lê o resultado da última ferramenta e escolhe a próxima, num laço que só termina quando ele julga a tarefa concluída. Num workflow, a ordem dos passos é fixada de antemão na definição, e um modelo de linguagem, quando aparece, é mais uma aplicação invocada — classifica um texto ou redige uma resposta, e o restante do processo continua determinado. A diferença decide quando usar cada um: o workflow serve a processos em que a sequência é conhecida e cada erro precisa ser rastreável até um passo; o agente, a tarefas em que o caminho só se descobre durante a execução. Nas próximas aulas iremos combinar o agente com o workflow.

---

3. O n8n

3.1 Características

O **n8n** é uma plataforma de automação de workflows criada em 2019 por Jan Oberhauser, em que a definição do processo é desenhada numa tela como um grafo de **nós** e executada por um motor próprio, escrito em TypeScript e executado pelo Node.js. Cada nó realiza uma operação — receber um gatilho, chamar um serviço, transformar dados ou decidir um caminho —, e a versão 2.42.5 traz 921 tipos de nós, entre integrações com serviços, nós de lógica e nós de inteligência artificial. Os componentes da seção 2.1 têm correspondentes diretos: o editor é a ferramenta de definição, o motor

é o serviço de execução, os nós de integração acionam as aplicações invocadas e a área Executions cumpre o papel de monitoramento.

3.2 Licença

O código do n8n é aberto para leitura, mas a licença não é de software livre no sentido estrito. A maior parte do código é distribuída sob a **Sustainable Use License**, que permite usar e modificar o software para fins internos da própria organização ou para uso não comercial, e proíbe revendê-lo como serviço; os arquivos marcados como empresariais exigem licença paga. O uso desta disciplina, numa instalação própria e sem revenda, está dentro do que a licença permite, e nenhum recurso pago é necessário para o que a aula constrói.

3.3 A linha 2 e os tutoriais antigos

A linha 2 do n8n trouxe mudanças de interface que tornam desatualizada boa parte dos tutoriais disponíveis na internet:

Antes (linha 1)	**Agora (linha 2)**
Interruptor **Active** no canto do editor	Botão **Publish**, com versões nomeadas (seção 4.5)
Botão **Save**	Salvamento automático do rascunho
Nó **Start**	Nó **Manual Trigger**
Nó Code com acesso às variáveis de ambiente	Nó Code isolado num *task runner*, sem acesso a elas

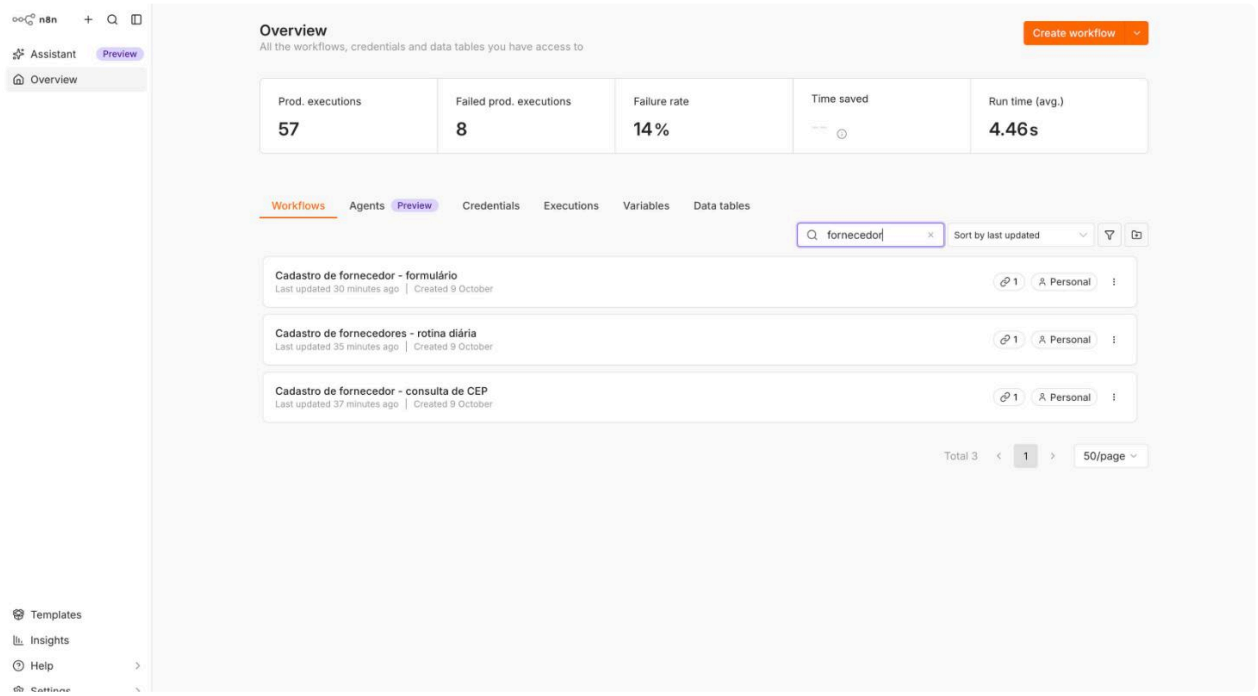
Quando a tela diferir do texto de um tutorial, a documentação oficial, listada nas Referências, é a fonte a consultar.

---

4. A interface do editor

4.1 Overview e lista de workflows

A tela inicial é a **Overview**, com a barra lateral à esquerda e a lista de workflows ao centro. A barra lateral dá acesso à Overview, ao espaço pessoal e, na parte inferior, a Templates, Insights, Help e Settings; o botão **Create workflow**, no alto à direita, abre um workflow novo. Acima da lista, as abas separam os workflows, as credenciais, as execuções, as variáveis e as **Data tables**, de modo que a mesma tela serve para localizar qualquer item da instalação.

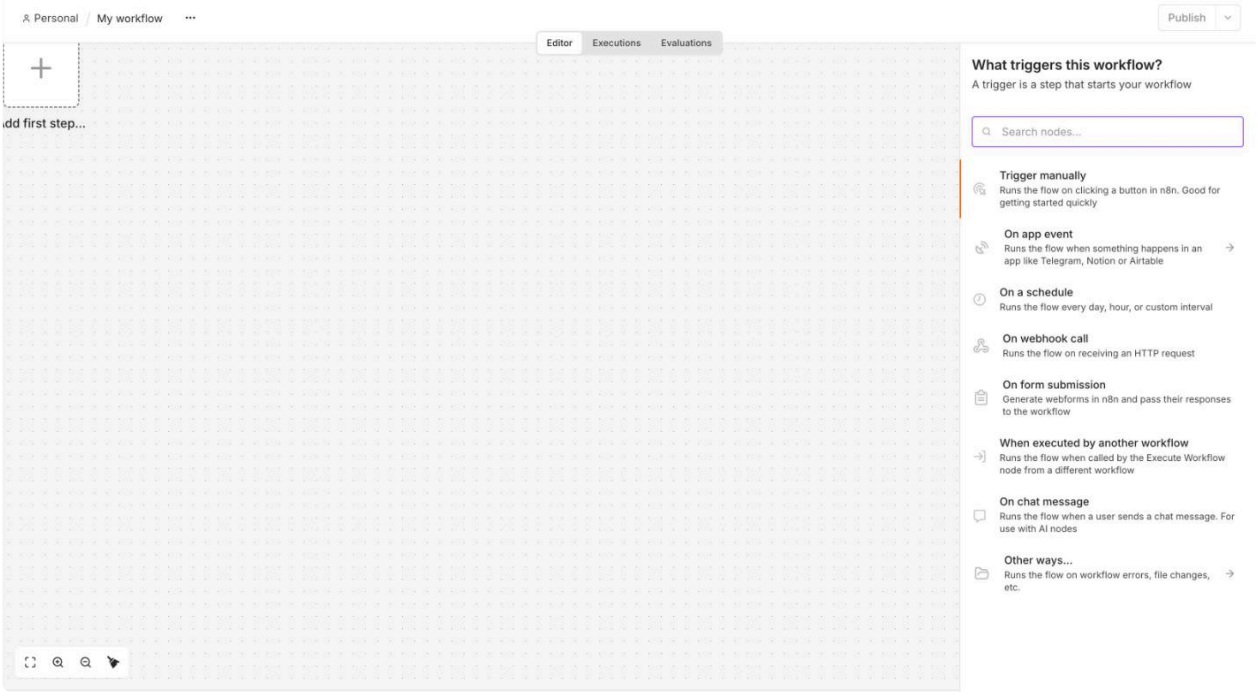


Fonte: Captura de tela do n8n 2.42.5, elaborada pelo autor.

A figura mostra a Overview depois desta aula, com os três workflows na lista. Cada workflow tem um espaço de origem — **Personal**, o espaço pessoal de quem o criou — e a indicação de publicação quando há uma versão em produção. Um workflow não publicado só roda quando alguém o executa no editor, ainda que tenha um gatilho de agenda ou de formulário, distinção tratada na seção 4.5.

4.2 O gatilho

Todo workflow começa por um **gatilho** (*trigger*), o nó que determina quando a execução acontece. Um workflow novo abre com o canvas vazio e o botão **Add first step…**, cujo clique abre o painel **What triggers this workflow?**:



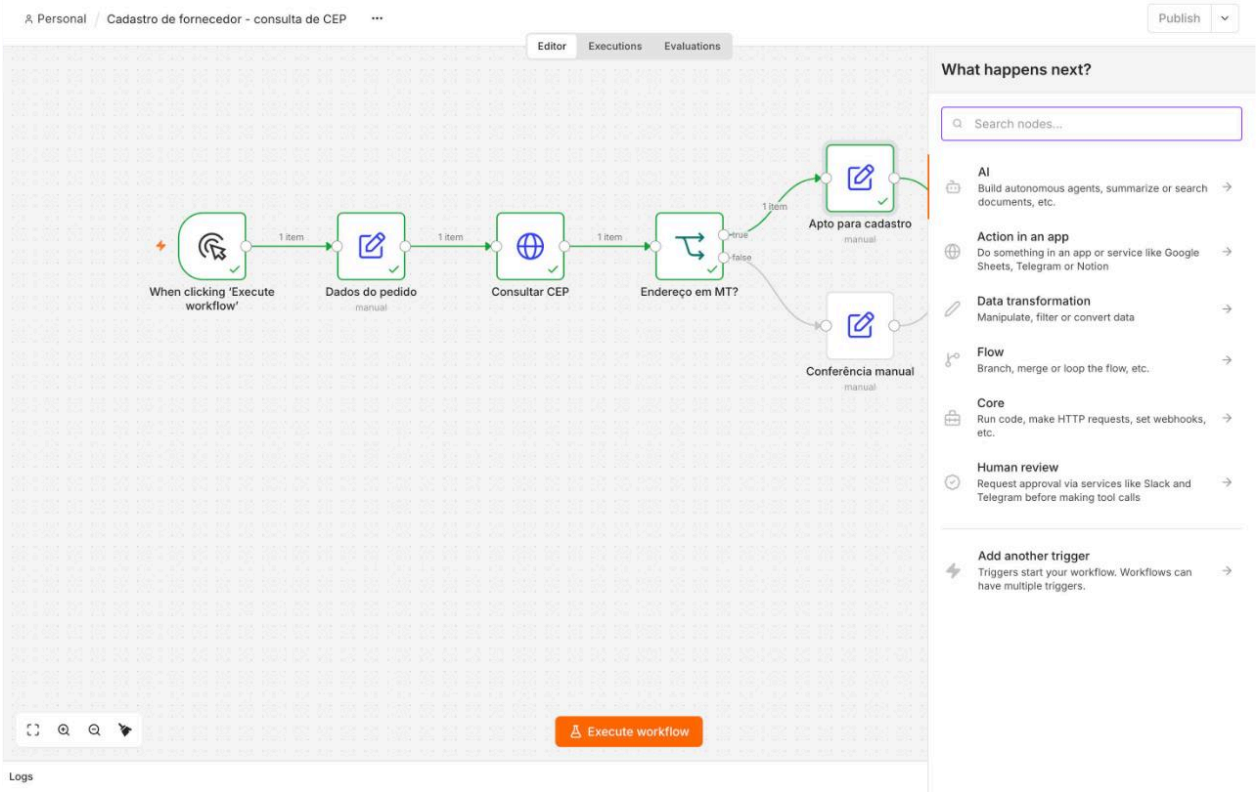
Fonte: Captura de tela do n8n 2.42.5, elaborada pelo autor.

Opção	Quando roda	Nesta aula
Trigger manually	Ao clicar em um botão do editor	Construção e teste, seção 6
On a schedule	Em horários definidos	Rotina diária, seção 7
On form submission	A cada envio de um formulário publicado pelo n8n	Variação da seção 7.5
On webhook call	Quando outro sistema faz uma requisição HTTP	Integração com outro sistema
On app event	Quando um serviço emite um evento, como um e-mail recebido	Aulas 10 a 12
On chat message	A cada mensagem num chat	Aula 10

O procedimento recomendado é construir e testar com o gatilho manual e dados fixos, e só depois trocar pelo gatilho de produção, com o restante dos nós inalterado — é o que a seção 7 faz. Um workflow pode ter mais de um gatilho, o que permite manter o manual para testes ao lado do de produção.

4.3 O painel de nós

Depois do gatilho, os nós são acrescentados pelo botão **+** à direita de cada nó, ou pelo botão **Open nodes panel**, no canto superior direito do canvas. O painel tem o campo **Search nodes...** e seis categorias:



Fonte: Captura de tela do n8n 2.42.5, elaborada pelo autor.

Categoria	Conteúdo
AI	Agentes, modelos e demais nós de inteligência artificial
Action in an app	Integrações com serviços, como Gmail, Google Sheets e Telegram
Data transformation	Nós que filtram, convertem e reorganizam dados
Flow	Nós que ramificam, juntam e repetem o fluxo
Core	Código, requisições HTTP e webhooks
Human review	Pedidos de aprovação humana antes de uma ação

A busca pelo nome é o caminho mais rápido quando o nó é conhecido. Dos 921 tipos de nós, a maior parte são integrações com serviços específicos; o núcleo usado em quase todo workflow é pequeno, e a seção 5.2 o apresenta.

4.4 Canvas, NDV e execução

No canvas, cada nó aparece como um cartão com ícone e nome, e as ligações saem da porta à direita de um nó para a porta à esquerda do seguinte. Um clique duplo no nó abre a **NDV** (*Node Details View*), com três painéis: à esquerda o **Input**, com os dados que chegaram ao nó; ao centro os parâmetros, nas abas **Parameters** e **Settings**; à direita o **Output**, com o que o nó produziu. Os painéis de entrada e saída mostram os dados nos formatos **Schema**, **Table** e **JSON**.

O botão **Execute workflow**, na parte inferior do canvas, roda o workflow inteiro a partir do gatilho manual, e o botão **Execute step**, dentro da NDV, roda apenas até o nó aberto, o que permite testar um passo por vez. Depois de uma execução, cada ligação exibe quantos itens trafegaram por ela, e cada nó ganha uma marca de sucesso ou de erro. O recurso **Pin data**, no painel de saída, fixa o resultado de um nó para que as execuções seguintes no editor o reutilizem sem refazer a chamada, o que evita consultar repetidamente um serviço externo durante a construção; os dados fixados valem só para execuções manuais e são ignorados em produção.

4.5 Salvamento, publicação e histórico

Na linha 2 não há botão de salvar: o editor grava as alterações automaticamente em poucos segundos, como **rascunho**. O rascunho pode ser executado no editor à vontade e não altera o que estiver em produção. Para colocar uma versão em produção, use o botão **Publish**, no canto superior direito, que abre a janela **Publish workflow** com campos para o nome da versão e a descrição das mudanças; enquanto o rascunho difere da versão publicada, o botão exibe um ponto amarelo.

A publicação é obrigatória para os gatilhos que dependem de evento externo: um Schedule Trigger não roda no horário marcado, e o formulário de um Form Trigger não

fica no ar, enquanto o workflow não for publicado. O menu ao lado do botão oferece **Unpublish**, que retira o workflow de produção, e o item **Version history** do menu do workflow permite comparar, restaurar e republicar versões anteriores. O cabeçalho do editor tem ainda as abas **Editor**, **Executions** e **Evaluations**; a aba Executions lista as execuções do workflow aberto e é usada na seção 7.4.

---

5. Itens, nós centrais e expressões

5.1 Itens

Os dados circulam entre os nós como uma lista de **itens**, e cada item é um objeto JSON com campos nomeados. Quando um nó recebe vários itens, ele executa sua operação uma vez para cada item, de modo que uma requisição HTTP que recebe cinco pedidos faz cinco consultas, sem laço explícito. Entender que cada nó trabalha item a item resolve a maior parte das dúvidas sobre o que o workflow está fazendo.

O n8n mantém também o vínculo entre cada item de saída e o item de entrada que o originou, chamado **item pareado** (*paired item*). É esse vínculo que permite, num nó adiante, recuperar a razão social da empresa que gerou uma resposta da API de CEP, embora a resposta não contenha a razão social; a seção 5.3 mostra a expressão que o utiliza.

5.2 Nós centrais

Um conjunto pequeno de nós aparece em quase todo workflow, e os desta aula usam a maior parte deles:

Nó	Categoria	Função	Uso nesta aula
Manual Trigger	Gatilho	Inicia a execução pelo botão Execute workflow	Teste de um pedido, seção 6
Schedule Trigger	Gatilho	Inicia em intervalos ou horários definidos	Rotina diária, seção 7.1
n8n Form Trigger	Gatilho	Publica um formulário e inicia a cada envio	Seção 7.5
Webhook	Gatilho	Inicia quando recebe uma requisição HTTP	Não usado nesta aula
Edit Fields (Set)	Data transformation	Cria, altera e remove campos dos itens	Dados do pedido e mensagem
HTTP Request	Core	Chama qualquer API pela web	Consulta do CEP
If	Flow	Envia cada item à saída true ou false conforme uma	Sede em Mato Grosso ou fora

Nó	Categoria	Função	Uso nesta aula
		condição	
Switch	Flow	Envia cada item a uma entre várias saídas	Não usado nesta aula
Code	Core	Executa JavaScript sobre os itens	Lista de pedidos do dia, seção 7.2
Data table	Data transformation	Lê e grava linhas numa tabela interna do n8n	Registro dos pedidos

O **Edit Fields (Set)** é o que mais aparece, porque prepara os dados no formato que o passo seguinte espera; ele tem os modos **Manual Mapping**, em que cada campo é definido por nome, tipo e valor, e **JSON**, e a opção **Include Other Input Fields** decide se os campos que chegaram são mantidos. O **Code** executa JavaScript num ambiente isolado, o *task runner*, e na linha 2 não tem acesso às variáveis de ambiente do servidor, o que impede que um código mal escrito leia chaves de API. O **Data table** grava em tabelas mantidas pelo próprio n8n, que dispensam uma planilha externa, com limite padrão de 200 MiB para o conjunto das tabelas da instalação.

5.3 Expressões

Qualquer parâmetro de nó recebe um valor fixo ou uma **expressão**, alternância feita pelos botões **Fixed** e **Expression** ao lado do campo. Uma expressão é um trecho de JavaScript entre chaves duplas, avaliado para cada item no momento da execução, e o editor mostra abaixo do campo o resultado calculado para o primeiro item, o que permite conferir a expressão antes de executar. Quatro formas cobrem quase todo o uso:

Expressão	Significado	Exemplo nesta aula
`{{ $json.campo }}`	Campo do item que chegou ao nó	`{{ $json.city }}`, o município devolvido
`{{ $('Nome do nó').item.json.campo }}`	Campo do item pareado num nó anterior	`{{ $('Dados do pedido').item.json.raz`
`{{ $now }}`	Data e hora atuais, como objeto Luxon	`{{ $now.setZone('America/Cuiaba').toForm HH:mm') }}`

Expressão	Significado	Exemplo nesta aula
`{{ $execution.id }}`	Número da execução atual	Protocolo do formulário, seção 7.5

A segunda forma usa o item pareado da seção 5.1 e é indispensável sempre que um nó substitui os dados que recebeu, como faz a requisição HTTP, que entrega a resposta da API no lugar do pedido. O nome entre parênteses é o nome do nó no canvas, exatamente como está escrito, e por isso renomear um nó depois de escrever as expressões quebra as que apontam para ele. A terceira forma usa a biblioteca Luxon, de manipulação de datas: `setZone` converte a hora para o fuso de Cuiabá e `toFormat` a escreve no formato brasileiro. Dentro das chaves vale qualquer expressão JavaScript, como o `replace(/\D/g, '')` da seção 6.3, mas uma lógica que ocupe mais de uma linha deve ir para um nó Code, onde pode ser lida e testada.

---

6. Primeiro workflow: consulta de CEP do fornecedor

O primeiro workflow processa um pedido, com gatilho manual. Ele é montado do zero, nó a nó; o arquivo `workflows/workflow01-consulta-cep.json` contém o resultado esperado e serve para conferência, não para ser importado no lugar da construção.

6.1 A Data table de registro

O registro dos pedidos substitui a planilha preenchida à mão, e por isso a tabela é criada antes do workflow. Na Overview, abra a aba **Data tables**, clique em **Create data table** e escolha **From scratch**. Dê à tabela o nome `cadastro_fornecedores` e crie onze colunas do tipo texto:

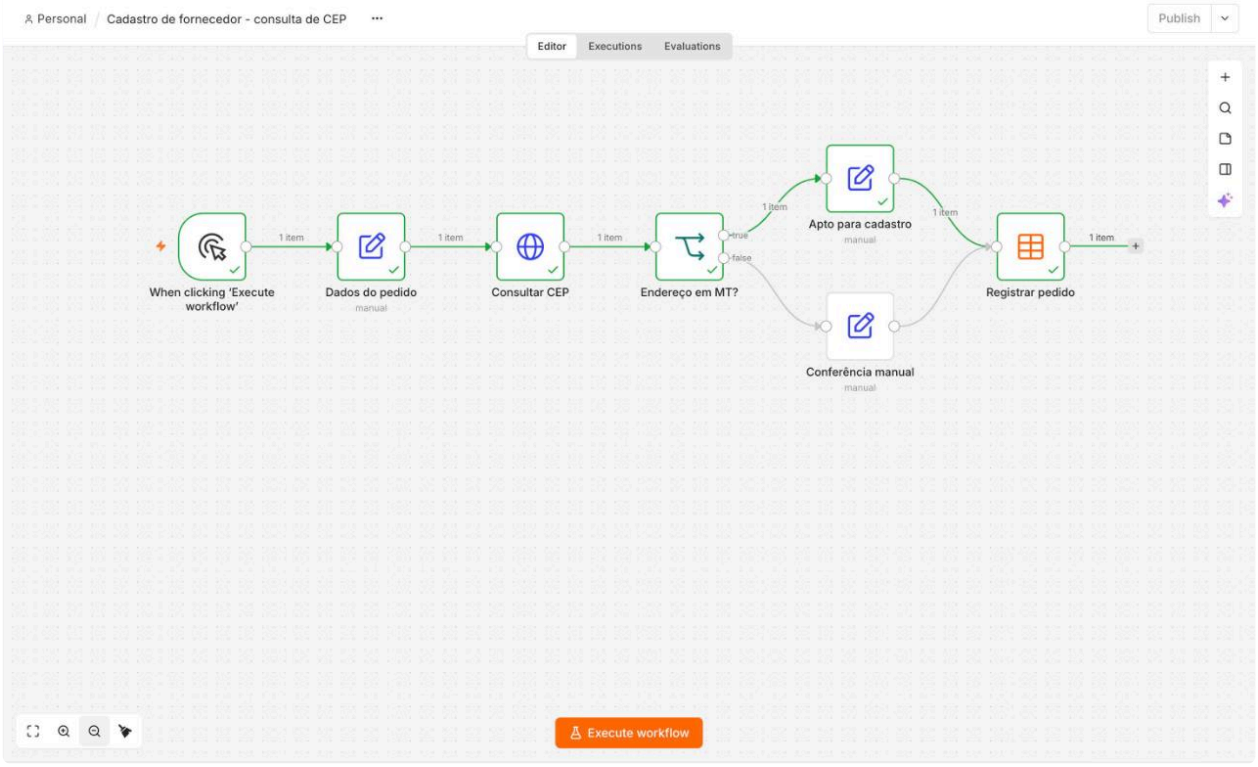
```
protocolo  razao_social  cnpj  email  cep  municipio  uf  logradouro
situacao  mensagem  consultado_em
```

Os nomes de coluna precisam começar por letra e conter apenas letras, números e sublinhado. A tabela acrescenta por conta própria as colunas de sistema `id`, `createdAt` e `updatedAt`. Verifique abrindo a tabela: devem aparecer as onze colunas, mais as três de sistema, e nenhuma linha.

6.2 Os sete nós

O workflow **Cadastro de fornecedor - consulta de CEP** tem sete nós. O gatilho manual entrega um item vazio ao nó **Dados do pedido**, que preenche os campos do pedido; **Consultar CEP** chama a BrasilAPI; **Endereço em MT?** decide o caminho;

Apto para cadastro e **Conferência manual** montam a linha e a mensagem de cada caminho; e **Registrar pedido** grava a linha na tabela.



Fonte: Captura de tela do n8n 2.42.5, elaborada pelo autor.

A figura mostra o workflow depois de uma execução, com um item em cada ligação percorrida. A saída false do If não recebeu item, porque o pedido de teste é de Cuiabá. O desenho reproduz três padrões da seção 2.2: a cadeia do gatilho até o If é uma sequência; o If é uma escolha exclusiva; e o nó Registrar pedido, que recebe as ligações dos dois caminhos, é uma junção simples.

Crie um workflow com **Create workflow** e escolha **Trigger manually**. O n8n dá ao gatilho o nome **When clicking 'Execute workflow'**; os demais nós devem ser renomeados à medida que forem criados, pelo item **Rename** do menu de contexto (botão direito sobre o nó) ou pela barra de espaço com o nó selecionado. Os nomes descritivos tornam o canvas legível e são usados nas expressões, por isso precisam ser escritos exatamente como acima, com acento e ponto de interrogação.

6.3 Dados do pedido e consulta à BrasilAPI

Acrescente um **Edit Fields (Set)**, renomeie-o para `Dados do pedido` e, no modo Manual Mapping, crie seis campos de texto com o pedido hipotético `PED-2026-0412`, copiado de `pedidos-fornecedores.json`, chave `pedido_unico`:

Campo	Valor
`protocolo`	`PED-2026-0412`
`razao_social`	`Papelaria Pantanal Comércio Ltda.`
`cnpj`	`47.021.358/0001-67`

Campo	Valor
`representante`	`Marta Aparecida Siqueira`
`email`	`contato@papelariapantanal.exemplo.com.br`
`cep`	`78048-000`

Todos os dados do arquivo são fictícios: os CNPJs não passam na verificação dos dígitos, para não corresponderem a empresa alguma, e os e-mails usam um domínio inexistente. Execute o nó com **Execute step** e verifique, no painel Output em Table, os seis campos com os valores da tabela.

A **BrasilAPI** é um serviço público e gratuito, mantido por voluntários, que reúne num só endereço consultas a CEP, CNPJ, bancos e feriados, sem cadastro nem chave. Antes de usá-la no workflow, confira a resposta pelo terminal:

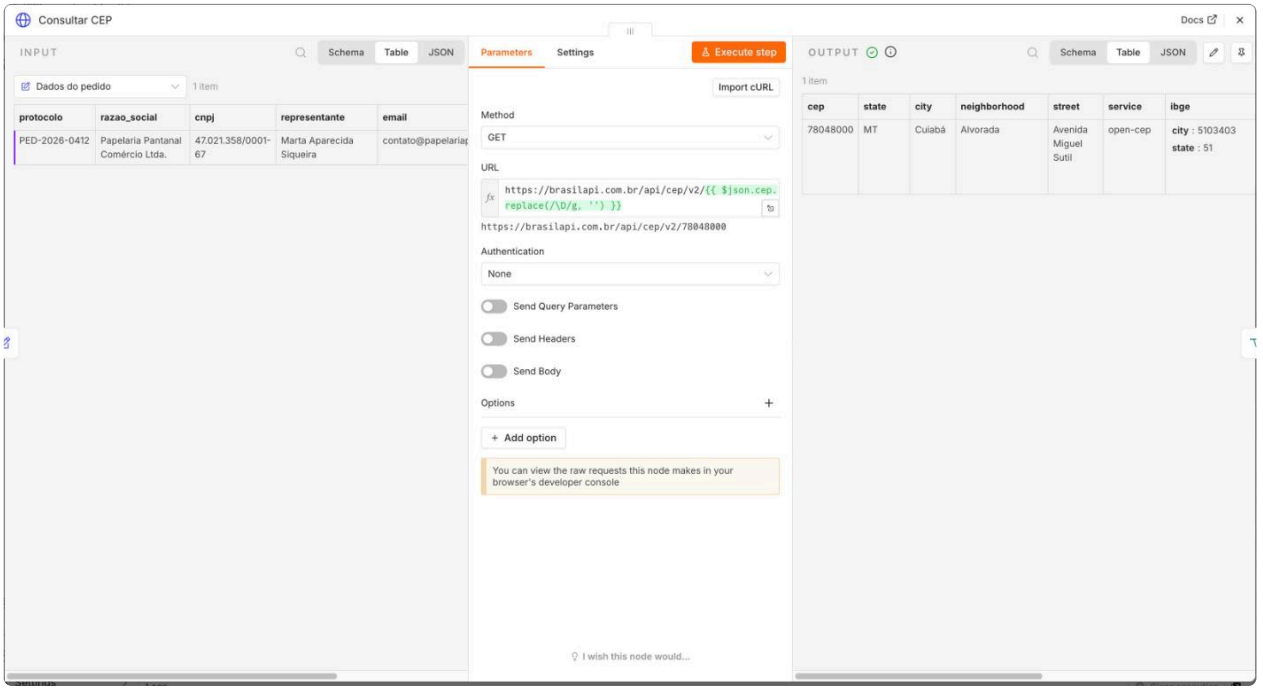
```
curl -s https://brasilapi.com.br/api/cep/v2/78048000
```

A resposta deve ser um objeto JSON com `"state":"MT"`, `"city":"Cuiabá"` e `"street":"Avenida Miguel Sutil"`, além do bairro, dos códigos do IBGE e das coordenadas.

Acrescente então um **HTTP Request**, renomeie-o para `Consultar CEP`, mantenha o método **GET** e escreva a URL como expressão:

```
https://brasilapi.com.br/api/cep/v2/{{ $json.cep.replace(/\D/g, '') }}
```

A expressão retira do CEP tudo o que não é dígito, porque o pedido traz `78048-000` e a API espera `78048000`. Execute o nó e compare o painel Output com a figura a seguir:



Fonte: Captura de tela do n8n 2.42.5, elaborada pelo autor.

O painel de saída contém apenas a resposta da API, sem a razão social nem o protocolo, que os nós seguintes recuperam pelo item pareado. A resposta obtida para o CEP do pedido de teste foi:

```
{
  "cep": "78048000",
  "state": "MT",
  "city": "Cuiabá",
  "neighborhood": "Alvorada",
  "street": "Avenida Miguel Sutil",
  "service": "open-cep",
  "ibge": { "city": "5103403", "state": "51" },
  "timezoneName": "America/Cuiaba"
}
```

Nas execuções feitas na preparação desta aula, a consulta à BrasilAPI levou de 0,13 a 2,7 segundos, e os demais nós, somados, menos de 0,1 segundo. A diferença indica onde estão o tempo e o risco do workflow: no serviço externo, que pode demorar, falhar ou mudar, e não no n8n. Por isso a seção 7.3 trata a falha da consulta como um caminho previsto do processo.

6.4 Decisão e mensagem

Acrescente um **If**, renomeie-o para `Endereço em MT?` e crie uma condição do tipo texto, operação **is equal to**, comparando `{{ $json.state }}` com `MT`. O If tem as saídas **true** e **false**, e cada item sai por uma delas. A regra hipotética do setor é que um fornecedor sediado em Mato Grosso segue direto para a análise documental, enquanto um sediado em outro estado envia o comprovante de endereço para conferência manual.

Ligue à saída true um Edit Fields chamado `Apto para cadastro` e à saída false outro chamado `Conferência manual`. Os dois montam a linha do registro com os mesmos onze campos da tabela da seção 6.1, e só mudam as colunas `situacao` e `mensagem`:

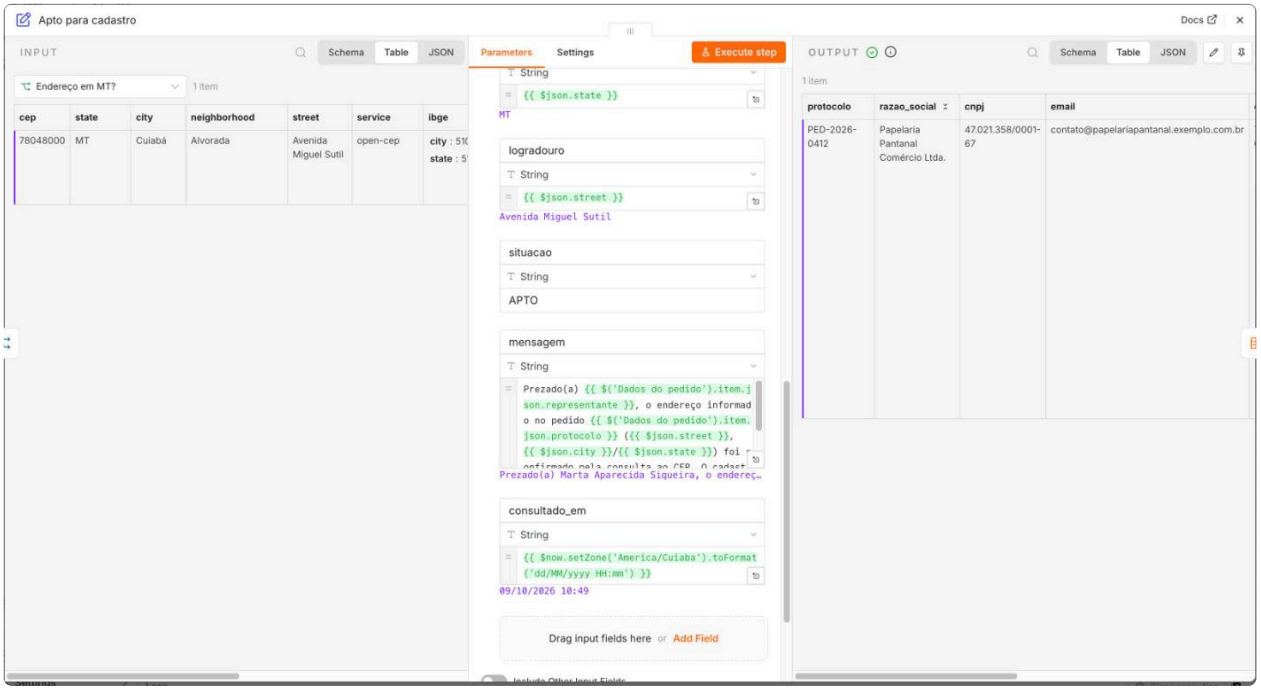
Coluna	Valor	Or
`protocolo`, `razao_social`, `cnpj`, `email`, `cep`	`{{ $('Dados do pedido').item.json.<campo> }}`	O pe pe ite pa
`municipio`, `uf`, `logradouro`	`{{ $json.city }}`, `{{ $json.state }}`, `{{ $json.street }}`	A res da
`situacao`	`APTO` ou `CONFERENCIA MANUAL`	Fix ca

Coluna	Valor	Or
		ca
`mensagem`	Texto com expressões, abaixo	Os
`consultado_em`	`{{ $now.setZone('America/Cuiaba').toFormat('dd/MM/yyyy HH:mm') }}`	Hc Cu

A mensagem de Apto para cadastro é um texto com expressões embutidas:

```
Prezado(a) {{ $('Dados do pedido').item.json.representante }}, o
endereço
informado no pedido {{ $('Dados do pedido').item.json.protocolo }}
({{ $json.street }}, {{ $json.city }}/{{ $json.state }}) foi
confirmado pela
consulta ao CEP. O cadastro da {{ $('Dados do
pedido').item.json.razao_social }}
segue para a análise documental.
```

A de Conferência manual segue o mesmo modelo e pede o envio do comprovante de endereço. Antes de executar, confira a prévia abaixo de cada campo, como na figura:



Fonte: Captura de tela do n8n 2.42.5, elaborada pelo autor.

A prévia mostra o valor calculado para o item antes mesmo de executar o nó; uma prévia em vermelho ou com `[undefined]` indica, quase sempre, nome de nó ou de campo escrito diferente do que está no canvas. A mensagem fica gravada na tabela e não é enviada a ninguém, porque o envio exige a credencial de um serviço de mensagens, assunto das Aulas 10 a 12; nesta aula, ela é um rascunho que uma pessoa confere antes de enviar.

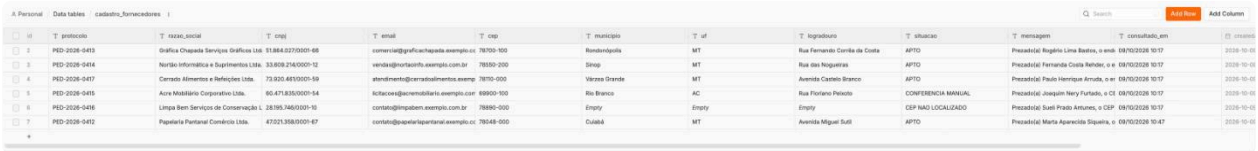
6.5 Registro e resultado

Acrescente um **Data table**, renomeie-o para `Registrar pedido` e ligue a ele as saídas dos dois Edit Fields. Configure o recurso **Row** e a operação **Insert**, e escolha a tabela no modo **By Name**, com o nome `cadastro_fornecedores`, para que o workflow continue funcionando quando importado em outra instalação, onde a tabela terá outro identificador. Em **Mapping Column Mode**, escolha **Map Automatically**, que grava cada campo do item na coluna de mesmo nome — é por isso que os Edit Fields usam exatamente os nomes das colunas.

Clique em **Execute workflow**. Todos os nós devem ganhar a marca de sucesso, com um item em cada ligação percorrida, e a tabela deve receber uma linha com estes valores:

```
protocolo     PED-2026-0412
razao_social  Papelaria Pantanal Comércio Ltda.
municipio     Cuiabá
uf            MT
logradouro    Avenida Miguel Sutil
situacao      APTO
mensagem      Prezado(a) Marta Aparecida Siqueira, o endereço
informado no
              pedido PED-2026-0412 (Avenida Miguel Sutil, Cuiabá/MT)
foi
              confirmado pela consulta ao CEP. O cadastro da Papelaria
              Pantanal Comércio Ltda. segue para a análise documental.
consultado_em <data e hora da execução>
```

Para conferir o outro caminho, troque o CEP do nó Dados do pedido por `69900-100`, de Rio Branco, e execute de novo. O item deve sair pela saída false do If, e a nova linha deve ter `uf` igual a `AC` e `situacao` igual a `CONFERENCIA MANUAL`. Volte o CEP para `78048-000` ao terminar.



Fonte: Captura de tela do n8n 2.42.5, elaborada pelo autor.

A figura mostra a tabela com os pedidos registrados ao fim da aula, separados pela coluna `situacao`. A tabela pode ser filtrada, editada à mão e baixada em CSV pela interface, e lida e gravada por outros workflows. Cada execução acrescenta uma linha, e executar duas vezes o mesmo pedido gera duas linhas iguais; a operação **Upsert** do mesmo nó, que atualiza a linha quando a chave já existe, é a forma de evitar essa duplicidade.

Exporte o workflow pronto com **Export JSON**, no menu do workflow (seção 8.1), salve-o como `workflows/meu-workflow01.json` e registre o commit:

```
git add workflows/meu-workflow01.json && git commit -m "Workflow de
consulta de CEP"
```

---

7. Workflow agendado: a rotina diária

O primeiro workflow processa um pedido por vez e depende de alguém clicar no botão. O segundo, **Cadastro de fornecedores - rotina diária**, processa sozinho, todos os dias às 8h, a lista de pedidos acumulados. Ele reaproveita os nós do primeiro e acrescenta três coisas: o gatilho de agenda, a lista de pedidos e o tratamento de erro.

7.1 Schedule Trigger e fuso horário

O gatilho é um **Schedule Trigger** renomeado para **Todo dia às 8h**, com **Trigger Interval** em **Days**, **Days Between Triggers** igual a 1 e **Trigger at Hour** igual a 8am. O mesmo campo oferece intervalos em segundos, minutos, horas, semanas e meses, e a opção **Custom (Cron)**, que aceita uma expressão no formato cron, herdado dos agendadores do Unix, como `0 8 * * 1-5` para as 8h dos dias úteis.

O horário é interpretado no fuso do workflow, e não no do computador. O fuso é definido no item **Settings** do menu do workflow, campo **Timezone**; quando o campo fica no padrão, vale o fuso da instalação, dado pela variável `GENERIC_TIMEZONE` da seção 1.2. Abra o Settings do workflow importado e verifique que o Timezone está em `America/Cuiaba`.

7.2 A lista de pedidos no nó Code

Em uso real, a lista viria do formulário da seção 7.5, de uma planilha ou de uma Data table de pedidos pendentes. Para que o workflow rode com dados hipotéticos, a lista do dia é produzida por um nó **Code**, chamado **Pedidos do dia**, que devolve um item por pedido:

```
// Lista dos pedidos recebidos no dia (dados hipotéticos).
// Em uso real, a lista viria do formulário, de uma planilha ou de uma
Data table.
const pedidos = [
  {
    "protocolo": "PED-2026-0413",
    "razao_social": "Gráfica Chapada Serviços Gráficos Ltda.",
    "cnpj": "51.864.027/0001-66",
    "representante": "Rogério Lima Bastos",
    "email": "comercial@graficachapada.exemplo.com.br",
```

```
    "cep": "78700-100"
  },
  {
    "protocolo": "PED-2026-0414",
    "razao_social": "Nortão Informática e Suprimentos Ltda.",
    "cnpj": "33.609.214/0001-12",
    "representante": "Fernanda Costa Rehder",
    "email": "vendas@nortaoinfo.exemplo.com.br",
    "cep": "78550-200"
  },
  {
    "protocolo": "PED-2026-0415",
    "razao_social": "Acre Mobiliário Corporativo Ltda.",
    "cnpj": "60.471.835/0001-54",
    "representante": "Joaquim Nery Furtado",
    "email": "licitacoes@acremobiliario.exemplo.com.br",
    "cep": "69900-100"
  },
  {
    "protocolo": "PED-2026-0416",
    "razao_social": "Limpa Bem Serviços de Conservação Ltda.",
    "cnpj": "28.195.746/0001-10",
    "representante": "Sueli Prado Antunes",
    "email": "contato@limpabem.exemplo.com.br",
    "cep": "78890-000"
  },
  {
    "protocolo": "PED-2026-0417",
    "razao_social": "Cerrado Alimentos e Refeições Ltda.",
    "cnpj": "73.920.461/0001-59",
    "representante": "Paulo Henrique Arruda",
    "email": "atendimento@cerradoalimentos.exemplo.com.br",
    "cep": "78110-000"
  }
];

return pedidos.map(pedido => ({ json: pedido }));
```

O nó Code devolve sempre uma lista de objetos com a chave `json`, que é a forma interna dos itens da seção 5.1. Os cinco pedidos foram escolhidos para percorrer todos os caminhos: três empresas com sede em Rondonópolis, Sinop e Várzea Grande, uma em Rio Branco, no Acre, e uma com um CEP que a BrasilAPI não localiza. Daí em diante, os nós são os do primeiro workflow, com as expressões apontando para `$('Pedidos do dia')` em vez de `$('Dados do pedido')`. Execute só este nó com Execute step e verifique cinco itens no painel Output.

7.3 Tratamento de erro na consulta

Quando a BrasilAPI não encontra um CEP, ela responde com o código HTTP 404 e a mensagem `Todos os serviços de CEP retornaram erro.`, o que se confere pelo terminal:

```
curl -s -o /dev/null -w "%{http_code}\n"
https://brasilapi.com.br/api/cep/v2/78890000
```

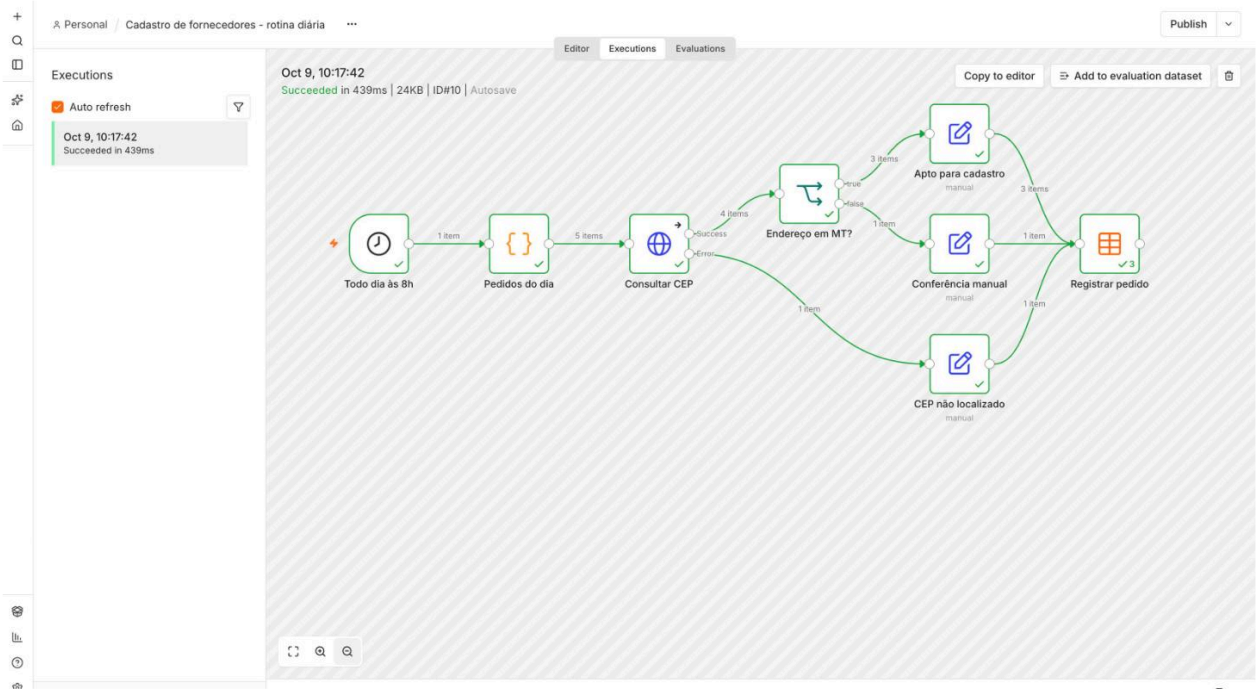
A resposta deve ser `404`. Por padrão, o nó HTTP Request interrompe o workflow inteiro diante desse erro, e, num processamento em lote, um único CEP digitado errado impediria o registro dos outros quatro pedidos. A correção está na aba **Settings** do nó Consultar CEP, campo **On Error**:

Opção	Comportamento
Stop Workflow	Interrompe o workflow inteiro; é o padrão
Continue	Passa o erro adiante como se fosse um resultado comum
Continue (using error output)	Cria uma segunda saída no nó, por onde saem só os itens que falharam

O workflow importado usa a terceira opção. O nó Consultar CEP passa a ter as saídas **Success** e **Error**, e a saída de erro leva ao Edit Fields **CEP não localizado**, que grava o pedido com a situação `CEP NAO LOCALIZADO`, as colunas de endereço vazias e uma mensagem que pede a correção do número. O item que sai pela saída de erro conserva os campos do pedido e acrescenta um campo `error` com a mensagem da API, de modo que o registro não se perde. A falha da consulta deixa de interromper o processo e passa a ser um caminho dele, com destino e mensagem próprios.

7.4 Publicação e execução de produção

O agendamento só funciona com o workflow publicado e o contêiner no ar. Para verificar a rotina sem esperar a manhã seguinte, mude temporariamente o Trigger Interval para **Minutes**, com intervalo 1, e clique em **Publish**. Acompanhe a aba **Executions**: em até um minuto deve aparecer uma execução de produção. Assim que ela aparecer, volte o gatilho para Days às 8am e publique de novo, para que a rotina não grave os mesmos cinco pedidos a cada minuto.



Fonte: Captura de tela do n8n 2.42.5, elaborada pelo autor.

A figura mostra uma execução de produção, com a situação e a duração na lista à esquerda e o caminho percorrido no canvas à direita: dos cinco pedidos, quatro saem pela saída Success e um pela Error do Consultar CEP, e o If separa os quatro em três de Mato Grosso e um de fora. Essa execução levou 0,44 segundo, e a contagem por nó, lida no registro da execução, foi a seguinte:

Nó	Itens recebidos	Itens por saída	Tempo
Todo dia às 8h	—	1	0 ms
Pedidos do dia	1	5	113 ms
Consultar CEP	5	4 em Success, 1 em Error	222 ms
Endereço em MT?	4	3 em true, 1 em false	3 ms
Apto para cadastro	3	3	50 ms
Conferência manual	1	1	11 ms
CEP não localizado	1	1	8 ms
Registrar pedido	5	3, 1 e 1, em três execuções	26 ms

Dois detalhes da tabela mostram como o motor funciona. As cinco consultas levaram, juntas, praticamente o tempo de uma só, porque o HTTP Request dispara as requisições dos itens sem esperar uma terminar para começar a outra; com listas grandes, a opção **Batching**, em Options, limita quantos itens são enviados por vez e o intervalo entre os lotes, o que evita sobrecarregar um serviço público. O nó Registrar pedido aparece executado três vezes, uma para cada ligação que chegou a ele: é a junção simples da seção 2.2, em que cada ramo grava seus itens quando termina, sem sincronização entre os ramos.

Verifique o resultado na tabela. Os pedidos `PED-2026-0413`, `PED-2026-0414` e `PED-2026-0417`, de Rondonópolis, Sinop e Várzea Grande, devem estar como `APTO`; o `PED-2026-0415`, de Rio Branco, como `CONFERENCIA MANUAL`; e o `PED-2026-0416`, cujo CEP `78890-000` a API não localiza, como `CEP NAO LOCALIZADO`. Em produção, essas execuções acontecem sem ninguém olhar, e a aba Executions é o lugar em que se confere, a cada manhã, se a rotina rodou e o que fez.

7.5 Variação com formulário

O cenário da seção 2 descreve pedidos que chegam por formulário, e o n8n tem um gatilho que publica o próprio formulário, o **n8n Form Trigger** (opção On form submission da seção 4.2). O workflow `workflows/workflow03-formulario-fornecedor.json` troca o gatilho manual por um Form Trigger com o título `Pedido de cadastro de fornecedor` e cinco campos obrigatórios: razão social, CNPJ, nome do representante, e-mail para contato e CEP da sede. Em cada campo, o parâmetro **Field Name** define o nome do campo no item, de modo que o formulário entrega `razao_social`, `cnpj`, `representante`, `email` e `cep`, e o nó Dados do pedido apenas repassa esses campos e **gera o protocolo** com a expressão:

```
PED-{{ $now.setZone('America/Cuiaba').toFormat('yyyy') }}-{{
$execution.id.padStart(4, '0') }}
```

O caminho do formulário é definido em **Options**, campo **Form Path**, como `cadastro-fornecedor`; sem esse campo, o endereço usa um identificador aleatório. Depois de publicado, o formulário fica em `http://localhost:5678/form/cadastro-fornecedor`. O endereço de teste, que só responde enquanto o editor está esperando uma execução, é o mesmo com `form-test` no lugar de `form`.

Num teste feito na instalação em contêiner, um envio com os dados de `pedido_formulario`, do arquivo de pedidos, percorreu o restante do workflow em 0,75 segundo e gravou na tabela um pedido `APTO` em Cuiabá, com protocolo `PED-2026-` seguido do número da execução. Na aba Executions, a execução aparece no modo `webhook`, e não `trigger`, porque o formulário chega ao n8n por uma requisição HTTP.

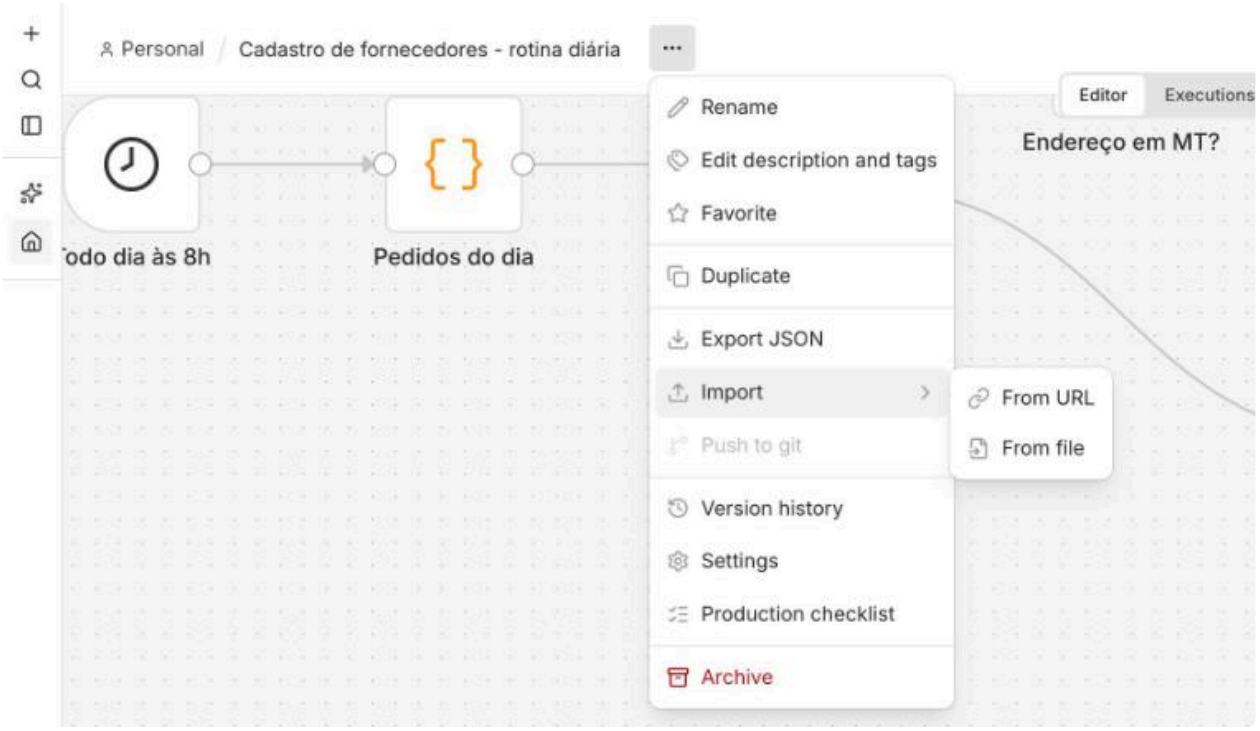
---

8. Workflows como arquivos: exportação, linha de comando e API

8.1 O arquivo JSON

Um workflow do n8n é um documento JSON, que pode ser gravado em arquivo, versionado no git e importado em outra instalação. O menu do workflow, aberto pelo botão de três pontos no cabeçalho do editor, tem a opção **Export JSON**, que baixa o

arquivo, e a opção **Import**, com os itens **From URL** e **From file**, que carregam um workflow no canvas aberto. Os nós também podem ser copiados com `Ctrl+C` e colados em outro workflow com `Ctrl+V`, porque o que vai para a área de transferência é o mesmo JSON.



Fonte: Captura de tela do n8n 2.42.5, elaborada pelo autor.

O arquivo tem cinco partes que importam:

Parte	Conteúdo
`nodes`	Cada nó, com nome, tipo, versão, posição no canvas e parâmetros
`connections`	Para cada nó de origem, a quais nós cada saída está ligada
`settings`	As configurações do workflow, como o fuso
`pinData`	Os dados fixados com Pin data
`id`	O identificador do workflow na instalação

O trecho a seguir, de `workflow02-rotina-diaria.json`, mostra as ligações do nó Consultar CEP; a primeira lista corresponde à saída Success, e a segunda à saída Error:

```
"Consultar CEP": {
  "main": [
    [ { "node": "Endereço em MT?", "type": "main", "index": 0 } ],
    [ { "node": "CEP não localizado", "type": "main", "index": 0 } ]
  ]
}
```

> ⚠️ **O que o arquivo exportado leva.** Ele não contém as senhas e chaves das credenciais, mas contém o nome e o identificador de cada credencial usada, e os dados fixados com Pin data, que podem incluir dados pessoais. Confira os dois pontos antes de publicar um workflow fora da sua máquina. Os três workflows desta aula não usam credenciais e foram exportados sem dados fixados.

8.2 Linha de comando

A imagem do n8n inclui o comando `n8n`, que, além de iniciar o servidor, lista, exporta e importa workflows. Como o servidor roda dentro do contêiner, o comando é executado lá dentro com `docker compose exec`, a partir da pasta `aula09`:

```
docker compose exec n8n n8n list:workflow
```

A saída lista um workflow por linha, no formato `id|nome`. Anote o `id` da rotina diária, usado nos próximos comandos.

A exportação escreve na saída padrão quando não recebe `--output`, e o redirecionamento grava o arquivo direto na pasta do projeto, sem depender de permissões dentro do contêiner. A opção `-T` desliga o terminal interativo, que misturaria caracteres de controle ao arquivo:

```
docker compose exec -T n8n n8n export:workflow --id=<id> --pretty >
workflows/rotina-cli.json
docker compose exec -T n8n n8n export:workflow --all >
workflows/todos.json
head -c 200 workflows/rotina-cli.json
```

O `head` deve mostrar o começo de uma lista JSON, `[`, seguida dos campos do workflow. A importação faz o caminho inverso: o arquivo é copiado para dentro do contêiner e importado lá:

```
docker compose cp workflows/workflow01-consulta-cep.json
n8n:/tmp/wf01.json
docker compose exec n8n n8n import:workflow --input=/tmp/wf01.json
```

A resposta deve terminar em `Successfully imported 1 workflow.`

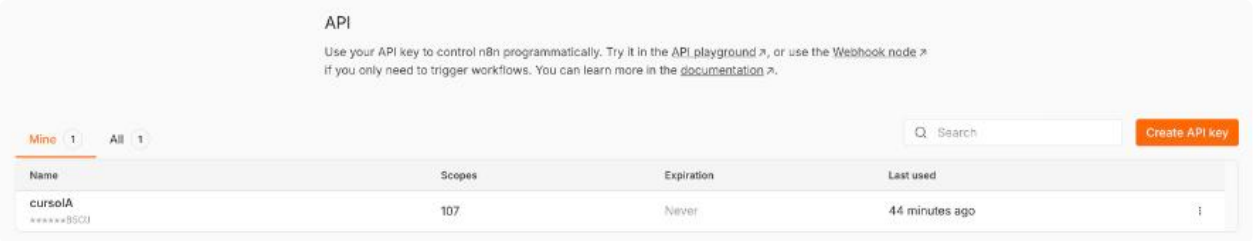
Três comportamentos da linha de comando, verificados na versão desta aula, condicionam o uso:

Comportamento	Consequência
O `export:workflow` grava uma lista JSON, mesmo para um só workflow, e inclui no	O arquivo não deve ser compartilhado como está; a

Comportamento	Consequência
campo `shared` o projeto dono, com nome e e-mail do usuário	operação `exportar` do cliente da seção 8.3 grava sem esse campo
O `import:workflow` exige o campo `id` no arquivo, e sobrescreve o workflow que já tiver o mesmo `id`	Importar duas vezes o mesmo arquivo não duplica o workflow, substitui
Alterações feitas pela linha de comando com o servidor no ar, como publicar um workflow, só têm efeito depois de reiniciá-lo	A linha de comando serve à cópia de segurança e à migração; a operação do dia a dia, com o servidor no ar, é feita pela API

8.3 API pública

A **API pública** do n8n permite listar, criar, publicar e apagar workflows, consultar execuções e ler e gravar Data tables a partir de qualquer programa. O acesso exige uma chave criada em **Settings**, página **n8n API**, botão **Create API key**, que abre uma janela com o rótulo da chave e a validade.



Fonte: Captura de tela do n8n 2.42.5, elaborada pelo autor.

A figura mostra a página com uma chave já criada: depois da criação, o valor fica mascarado, e as colunas mostram os escopos, a validade e o último uso. A chave só é exibida inteira uma vez, no momento da criação. Crie uma chave com o rótulo `aula09` e uma validade definida, que limita o dano de uma chave vazada, e guarde-a no `.env` da pasta do projeto:

```
cp .env.exemplo .env
# edite o .env e cole a chave depois de N8N_API_KEY=
git check-ignore .env
```

O `git check-ignore` deve responder `.env`, confirmando que o arquivo não entra nos commits.

> ⚠️ **A chave dá acesso à instalação inteira.** Com ela, qualquer programa lê os pedidos gravados, apaga workflows e cria outros. Ela fica só no `.env`, nunca no código, no `AGENTS.md` ou no `.zip` da entrega; se vazar, apague-a na mesma página e crie outra.

A chave vai no cabeçalho `X-N8N-API-KEY` de cada requisição, e o endereço base é `http://localhost:5678/api/v1/`. A instalação publica a documentação interativa em

`http://localhost:5678/api/v1/docs`, e os endereços usados nesta aula são estes:

Método e endereço	Função
`GET /workflows`	Listar os workflows
`POST /workflows`	Criar a partir de um JSON com `name`, `nodes`, `connections` e `settings`
`POST /workflows/{id}/publish` e `.../unpublish`	Publicar e retirar de produção
`GET /executions`	Ler as execuções
`POST /data-tables`	Criar uma Data table

A API não tem endereço para disparar um workflow de gatilho manual. Quem precisa disparar um workflow a partir de outro sistema usa o gatilho **Webhook**, cuja URL de produção, `http://localhost:5678/webhook/<caminho>`, passa a responder assim que o workflow é publicado.

O `cliente_n8n.py` da pasta reúne as operações que se fazem sem abrir a tela, com a chave lida do `.env`:

```
URL = os.environ.get("N8N_URL", "http://127.0.0.1:5678") + "/api/v1"
cliente = httpx.Client(base_url=URL, headers={"X-N8N-API-KEY":
os.environ["N8N_API_KEY"]}, timeout=60)


def importar(arquivo: str) -> None:
    """Cria um workflow a partir de um arquivo JSON exportado."""
    dados = json.loads(Path(arquivo).read_text(encoding="utf-8"))
    if isinstance(dados, list):  # o export:workflow da linha de
comando grava uma lista
        dados = dados[0]
    corpo = {k: dados[k] for k in CAMPOS_GRAVAVEIS if k in dados}
    r = cliente.post("/workflows", json=corpo)
    r.raise_for_status()
    print(f"importado: {r.json()['name']} ({r.json()['id']})")
```

As operações são `listar`, `importar`, `exportar` (que grava o workflow sem o projeto dono e sem os identificadores de credencial), `execucoes` (que lista as execuções de um workflow com a duração e os itens por nó), `criar-tabela` e `linhas`. Com o ambiente da seção 1.4 ativo, verifique a chave listando os workflows:

```
python cliente_n8n.py listar
```

A saída deve mostrar um workflow por linha, com o `id`, a situação (`publicado` ou `rascunho`) e o nome. A consulta às execuções da rotina diária devolve o mesmo

resumo que a aba Executions mostra:

```
python cliente_n8n.py execucoes <id da rotina>
```

```
execução 10  modo=trigger  situação=success  2026-10-09T14:17:42.025Z
0.44 s
    Todo dia às 8h                    0 ms  itens por saída: 1
    Pedidos do dia                  113 ms  itens por saída: 5
    Consultar CEP                   222 ms  itens por saída: 4/1
    Endereço em MT?                   3 ms  itens por saída: 3/1
    Apto para cadastro               50 ms  itens por saída: 3
    Registrar pedido                 10 ms  itens por saída: 3
    Registrar pedido                  4 ms  itens por saída: 1
    Registrar pedido                 12 ms  itens por saída: 1
    Conferência manual               11 ms  itens por saída: 1
    CEP não localizado                8 ms  itens por saída: 1
```

O campo `modo` distingue a origem de cada execução, o que permite separar o que rodou sozinho do que alguém disparou: `trigger` indica os agendamentos, `webhook` as chamadas HTTP e os formulários, `manual` as execuções feitas no editor e `cli` as da linha de comando. Os arquivos de `workflows/` podem ser importados em outra instalação por qualquer dos três caminhos — o menu Import da interface, o `import:workflow` da linha de comando ou o `importar` do cliente —, desde que a Data table `cadastro_fornecedores` exista antes; a operação `criar-tabela` a cria com as onze colunas da seção 6.1.

8.4 O agente e o workflow

Um workflow em arquivo JSON e uma API com saída textual são o mesmo arranjo que a Aula 06, seção 2, usou com o `arduino-cli`: o agente lê e edita o arquivo, executa o `cliente_n8n.py` e lê a resposta, sem que seja preciso um servidor MCP. O pedido ao agente segue o formato da Aula 02, seção 9.2, e o critério de aceite é verificável pelo próprio cliente — por exemplo, acrescentar uma coluna ao workflow e conferir, com `python cliente_n8n.py linhas cadastro_fornecedores`, que a linha nova a preenche. O que muda em relação ao firmware é o risco: a chave da API dá acesso à instalação inteira, e por isso o `AGENTS.md` deve proibir a leitura do `.env` e qualquer operação de apagar.

---

9. Governança de um workflow em uso

Os workflows desta aula tratam dados de pessoas desde o primeiro nó, porque o pedido traz o nome e o e-mail do representante da empresa, e a Lei Geral de Proteção de Dados Pessoais (LGPD, Lei nº 13.709/2018) se aplica a esses dados mesmo

quando o titular age em nome de uma pessoa jurídica. O n8n grava esses dados em três lugares, e cada um pede uma decisão de quem opera a instalação:

Lugar	O que guarda	Decisão
Data table	O registro desejado	A mesma política de retenção da planilha que ela substitui
Execuções	A entrada e a saída de cada nó, em cada execução	Por quanto tempo guardar e quais execuções guardar
Credenciais	As chaves dos serviços, cifradas	Quem tem acesso à chave de cifra

As execuções são o ponto menos visível. Por padrão, a instalação grava todas as execuções, manuais e de produção, bem-sucedidas ou não, com os dados completos de cada nó, e apaga automaticamente as que têm mais de 336 horas (14 dias) ou que excedem 10 mil execuções; esses limites mudam pelas variáveis `EXECUTIONS_DATA_MAX_AGE` e `EXECUTIONS_DATA_PRUNE_MAX_COUNT`, acrescentadas ao `environment` do `docker-compose.yml`. Nas configurações de cada workflow, os campos **Save successful production executions** e **Save manual executions** permitem não gravar as execuções bem-sucedidas, mantendo só as que falharam, o que reduz o acúmulo de dados pessoais sem perder o diagnóstico dos erros.

Tudo isso fica no volume `n8n_data`, montado em `/home/node/.n8n`: o banco SQLite, em texto legível, e o arquivo `config`, com a chave que cifra as credenciais. A consequência é dupla. A cópia de segurança precisa incluir o arquivo `config`, porque sem ele as credenciais restauradas ficam ilegíveis; e quem tem acesso ao volume tem acesso a todos os pedidos, de modo que a máquina que hospeda o n8n precisa do mesmo controle de acesso dado a qualquer sistema com dados pessoais. O comando `docker compose down -v` apaga o volume e, com ele, todos os workflows, tabelas e execuções.

O serviço externo também exige critério. A BrasilAPI é gratuita, mantida por voluntários e não oferece garantia de disponibilidade, e por isso a consulta de CEP é um dado auxiliar, que confirma o endereço declarado, e não uma prova. A mesma API oferece a consulta de CNPJ, em `https://brasilapi.com.br/api/cnpj/v1/<cnpj>`, que devolve os dados públicos do cadastro da empresa, inclusive o quadro de sócios com nomes de pessoas físicas; essa consulta, quando acrescentada ao workflow, amplia os dados pessoais que as execuções passam a guardar.

Por fim, a mensagem que o workflow redige é um rascunho, e não uma comunicação oficial. O workflow confirma um endereço e aplica uma regra simples, mas a decisão sobre o cadastro continua com a pessoa que analisa os documentos, e a mensagem só deve sair depois dessa análise. As Aulas 10 a 12 acrescentam envio de mensagens e modelos de linguagem, e a mesma separação entre o que o workflow prepara e o que a pessoa decide continua valendo.

10. Tropeços comuns

Sintoma	Causa	Solução
`docker: unknown command: docker compose`	O plugin do Compose não está instalado ou não foi registrado no Docker	Instalar e registrar o plugin, como na nota da seção 1.2; conferir com `docker compose version`
`service "n8n" is not running` no `docker compose exec`	O contêiner não foi iniciado a partir desta pasta, ou o n8n em uso foi instalado pelo npm	Rodar `docker compose up -d` na pasta `aula09`; quem usa o npm digita o comando sem o prefixo, `n8n list:workflow`
`port is already allocated` no `docker compose up`	Outro processo já usa a porta 5678, como um n8n instalado pelo npm	Parar o outro processo, ou trocar o mapeamento para `"127.0.0.1:5680:5678"` e acessar `http://localhost:5680`
O `curl` da seção 1.2 não responde	O n8n ainda está iniciando	Aguardar alguns segundos e acompanhar com `docker compose logs -f n8n`
O agendamento roda uma hora antes ou depois	Fuso do workflow ou da instalação diferente de Cuiabá	Conferir o Timezone em Settings do workflow e o `GENERIC_TIMEZONE` do `docker-compose.yml`
O Schedule Trigger não roda no horário	Workflow não publicado, ou contêiner parado	Publicar o workflow (seção 4.5) e conferir `docker compose ps`
O nó Registrar pedido falha ao importar o workflow	A Data table não existe na instalação, ou tem outro nome	Criar `cadastro_fornecedores` com as onze colunas (seção 6.1) ou com `python cliente_n8n.py criar-tabela cadastro_fornecedores`
Prévia da expressão em vermelho ou `[undefined]`	O nome entre `$('...')` difere do nome do nó no canvas	Copiar o nome exato do nó, com acentos e pontuação
Um CEP inválido interrompe a rotina inteira	**On Error** em Stop Workflow, o padrão	Usar **Continue (using error output)** no Consultar CEP (seção 7.3)
O endereço `/form/cadastro-fornecedor` responde 404	Workflow do formulário não publicado	Publicar; para testar sem publicar, usar `/form-test/` com o editor aguardando a execução
`ENOENT: no such file or directory`	O arquivo está no computador, e o	Copiar antes com `docker compose cp` (seção 8.2)

Sintoma	Causa	Solução
no `import:workflow`	comando roda dentro do contêiner	
`KeyError: 'N8N_API_KEY'` no `cliente_n8n.py`	O `.env` não existe ou não tem a chave	Criar o `.env` a partir do `.env.exemplo` (seção 8.3)
`401 Unauthorized` no `cliente_n8n.py`	Chave errada, apagada ou vencida	Criar outra chave em Settings › n8n API

Laboratório e entrega

A atividade é em grupo, com uma instalação do n8n por grupo, e transforma o roteiro de consulta e registro desta aula num processo em produção, com agendamento e documentação de onde ficam os dados que ele trata. Todo o trabalho fica na pasta `aula09` da seção 1.1, com um commit ao fim de cada parte.

O que entregar

O `.zip` deve conter:

1. `docker-compose.yml`
2. Em `workflows/`, os workflows do grupo exportados ao fim das Partes 2 a 4: o de consulta de CEP com as colunas e a mensagem dos passos 6 e 7, e a rotina diária publicada;
3. Link do Git

Referências

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). **Diário Oficial da União**, Brasília, DF, 15 ago. 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: out. 2026.

BRASILAPI. **BrasilAPI**: documentação. Disponível em: https://brasilapi.com.br/docs. Acesso em: out. 2026.

HOHPE, G.; WOOLF, B. **Enterprise integration patterns**: designing, building, and deploying messaging solutions. Boston: Addison-Wesley, 2003.

HOLLINGSWORTH, D. **The workflow reference model**. Winchester: Workflow Management Coalition, 1995. (Document TC00-1003, Issue 1.1).

N8N. 2.0 breaking changes. **n8n Docs**. Disponível em: https://docs.n8n.io/2-0-breaking-changes/. Acesso em: out. 2026.

N8N. Authentication. **n8n Docs**. Disponível em: https://docs.n8n.io/api/authentication/. Acesso em: out. 2026.

N8N. CLI commands. **n8n Docs**. Disponível em: https://docs.n8n.io/hosting/cli-commands/. Acesso em: out. 2026.

N8N. Data tables. **n8n Docs**. Disponível em: https://docs.n8n.io/data/data-tables/. Acesso em: out. 2026.

N8N. Docker installation. **n8n Docs**. Disponível em: https://docs.n8n.io/hosting/installation/docker/. Acesso em: out. 2026.

N8N. Export and import workflows. **n8n Docs**. Disponível em: https://docs.n8n.io/workflows/export-import/. Acesso em: out. 2026.

N8N. Expressions. **n8n Docs**. Disponível em: https://docs.n8n.io/code/expressions/. Acesso em: out. 2026.

N8N. npm installation. **n8n Docs**. Disponível em: https://docs.n8n.io/hosting/installation/npm/. Acesso em: out. 2026.

N8N. Publish workflows. **n8n Docs**. Disponível em: https://docs.n8n.io/workflows/publish/. Acesso em: out. 2026.

N8N. Schedule Trigger node. **n8n Docs**. Disponível em: https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.scheduletrigger/. Acesso em: out. 2026.

N8N-IO. **n8n**: LICENSE.md. GitHub. Disponível em: https://github.com/n8n-io/n8n/blob/master/LICENSE.md. Acesso em: out. 2026.

VAN DER AALST, W. M. P.; BICHLER, M.; HEINZL, A. Robotic process automation. **Business & Information Systems Engineering**, v. 60, n. 4, p. 269-272, 2018.

VAN DER AALST, W. M. P.; TER HOFSTEDE, A. H. M.; KIEPUSZEWSKI, B.; BARROS, A. P. Workflow patterns. **Distributed and Parallel Databases**, v. 14, n. 1, p. 5-51, 2003.

WORKFLOW MANAGEMENT COALITION. **Terminology & glossary**. Winchester: WfMC, 1999. (Document WFMC-TC-1011, Issue 3.0).

Aula anterior: `Aula-08-Docker-e-Publicacao-ghcr.md` · **Próxima aula:** `Aula-10-ChatBot-Multiagente-n8n.md`